000 — Pre-Bootcamp Setup Guide

> ⚠️ **IMPORTANT:** Complete this guide **BEFORE** Saturday's bootcamp. Estimated time: **45–90 minutes** (depending on internet speed and OS).
>
> If you get stuck, screenshot the error and send it to the group chat.

Table of Contents

---

1. Create Required Accounts

1.1 GitHub Account

If you don't already have one:

1. Go to https://github.com/signup
2. Create your account with a professional username (this will appear in your deployment URLs)
3. **Verify your email address** (check inbox for verification email)
4. Once created, send your GitHub username to the instructor so we can invite you to the organization

> 📌 **Note:** After receiving the organization invite, accept it at: https://github.com/orgs/proxeon/invitation

1.2 DigitalOcean Account

1. Go to https://www.digitalocean.com
2. Sign up (you can use your GitHub account to sign in)
3. **Add a payment method** (credit/debit card required — you won't be charged yet)
4. If available, use this referral link for free credits: `[INSTRUCTOR WILL PROVIDE LINK]`

> 💰 **Cost expectation:** The resources we create will cost approximately **$6–12/month**. You can destroy everything after the bootcamp if you don't want to keep it running.

---

2. Install Development Tools (Windows)

> 🖥️ **Skip this section if you're on macOS.** Jump to Section 3.

2.1 System Requirements

- Windows 10 version 2004+ (Build 19041+) or Windows 11
- At least **8GB RAM** (16GB recommended)
- **Virtualization enabled** in BIOS/UEFI

Check your Windows version: Press `Win + R`, type `winver`, press Enter. Your build number must be **19041 or higher**.

2.2 Install WSL2

Open **PowerShell as Administrator** (right-click Start → "Windows Terminal (Admin)" or "PowerShell (Admin)"):

```
wsl --install
```

This command will:

- Enable WSL feature
- Enable Virtual Machine Platform
- Download and install Ubuntu (default)

> ⚠️ **You MUST restart your computer after this step.**

After restart, Ubuntu will open automatically and ask you to create a username and password. These are your Linux credentials (not your Windows password).

Verify WSL2 is working:

```
wsl --list --verbose
```

You should see:

```
  NAME      STATE           VERSION
* Ubuntu    Running         2
```

> ⚠️ If VERSION shows `1` instead of `2`, run:
>
> ```
> wsl --set-version Ubuntu 2
> ```

2.3 Install Docker Engine Inside WSL

> ⚠️ **We install Docker inside WSL2, NOT Docker Desktop.** Docker Desktop is heavy and has licensing restrictions. Docker Engine inside WSL is lighter and free.

Open your **Ubuntu terminal** (search "Ubuntu" in Start menu):

```
# Update package lists
sudo apt update && sudo apt upgrade -y

# Install prerequisites
sudo apt install -y ca-certificates curl gnupg lsb-release

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin

# Add your user to the docker group (so you don't need sudo)
sudo usermod -aG docker $USER
```

Now close and reopen your Ubuntu terminal (this applies the group change).

Start Docker daemon:

```
sudo service docker start
```

> 💡 **Tip:** Docker daemon doesn't auto-start in WSL2. You'll need to run `sudo service docker start` each time you open a new terminal, OR add it to your `~/.bashrc`:

```
echo '# Auto-start Docker daemon
if service docker status 2>&1 | grep -q "is not running"; then
    sudo service docker start >/dev/null 2>&1
fi' >> ~/.bashrc
```

Then configure passwordless sudo for docker service:

> ```
> sudo visudo -f /etc/sudoers.d/docker
> ```
>
> Add this line (replace `YOUR_USERNAME` with your actual WSL username):
>
> ```
> YOUR_USERNAME ALL=(ALL) NOPASSWD: /usr/sbin/service docker *
> ```
>
> Save with `Ctrl+X`, then `Y`, then `Enter`.

Verify Docker installation:

```
docker --version
docker compose version
docker run hello-world
```

You should see a "Hello from Docker!" message. ✅

2.4 Install Git (inside WSL)

Git usually comes pre-installed with Ubuntu. Verify:

```
git --version
```

If not installed:

```
sudo apt install -y git
```

2.5 Important: Always Use WSL Terminal

For the rest of this guide and during the bootcamp, **always use the Ubuntu/WSL terminal**, not PowerShell or CMD.

You can access WSL from Windows Terminal by clicking the dropdown arrow and selecting "Ubuntu".

---

3. Install Development Tools (macOS)

> 🍎 **Skip this section if you're on Windows.** You should have completed Section 2.

3.1 Install Homebrew (if not already installed)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

After installation, follow the instructions printed in the terminal to add Homebrew to your PATH.

3.2 Install Docker Desktop for macOS

1. Go to https://www.docker.com/products/docker-desktop/
2. Download the version for your chip:
   - **Apple Silicon (M1/M2/M3/M4):** "Mac with Apple chip"
   - **Intel Mac:** "Mac with Intel chip"
3. Open the `.dmg` file and drag Docker to Applications
4. Launch Docker Desktop from Applications
5. Accept the terms and **wait for Docker to fully start** (whale icon in menu bar stops animating)

Verify Docker installation:

```
docker --version
docker compose version
docker run hello-world
```

3.3 Install Git

```
# Git comes with Xcode Command Line Tools
xcode-select --install
```

Or via Homebrew:

```
brew install git
```

Verify:

```
git --version
```

---

4. Generate SSH Key

> This SSH key will be used to:
>
> 1. Authenticate with GitHub (push/pull code)
> 2. Connect to your DigitalOcean VPS

> We'll generate **one key** and use it for both purposes.

4.1 Generate the Key

Run this in your terminal (**WSL Ubuntu terminal** for Windows, **Terminal.app** for macOS):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Replace `your_email@example.com` with the email you used for GitHub.

When prompted:

- **File location:** Press `Enter` to accept default (`~/.ssh/id_ed25519`)
- **Passphrase:** Press `Enter` for no passphrase (or set one if you prefer)

4.2 Start SSH Agent and Add Key

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

4.3 Copy Your Public Key

```
cat ~/.ssh/id_ed25519.pub
```

Copy the entire output (starts with `ssh-ed25519` and ends with your email). You'll need this in the next steps.

> 💡 **Windows/WSL Tip:** To copy from WSL terminal, select the text and right-click (or `Ctrl+Shift+C`).

---

5. Add SSH Key to GitHub

1. Go to https://github.com/settings/ssh/new
2. **Title:** Enter something like `Bootcamp Laptop` or `My MacBook`
3. **Key type:** Authentication Key
4. **Key:** Paste the public key you copied in step 4.3
5. Click **Add SSH key**

Test the connection:

```
ssh -T git@github.com
```

You should see:

```
Hi YOUR_USERNAME! You've successfully authenticated, but GitHub does not
provide shell access.
```

> ⚠️ If you see "Permission denied (publickey)", see Troubleshooting.

---

6. Configure Git Identity

Set your name and email for Git commits:

```
git config --global user.name "Your Full Name"
git config --global user.email "your_email@example.com"
```

Set default branch name:

```
git config --global init.defaultBranch main
```

Verify:

```
git config --global --list
```

---

7. Install Additional Tools

These tools will be needed during the bootcamp for local development.

7.1 Install Node.js (v20 LTS)

macOS:

```
brew install node@20
```

WSL/Ubuntu:

```
# Install Node.js 20 via NodeSource
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

Verify:

```
node --version   # Should show v20.x.x
npm --version    # Should show 10.x.x
```

7.2 Install pnpm

```
npm install -g pnpm
```

Verify:

```
pnpm --version   # Should show 9.x.x or higher
```

7.3 Install .NET SDK 10 (Optional — only needed for local API development)

> 📌 **Note:** This is optional for the bootcamp. The API runs inside Docker during deployment. Only install if you want to run the .NET API locally outside of Docker.

macOS:

```
brew install dotnet-sdk
```

WSL/Ubuntu:

```
# Add Microsoft package repository
wget https://packages.microsoft.com/config/ubuntu/$(lsb_release -
rs)/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
rm packages-microsoft-prod.deb

sudo apt update
sudo apt install -y dotnet-sdk-10.0
```

Verify:

```
dotnet --version   # Should show 10.x.x
```

---

8. Verification Checklist

Run this **single command** in your terminal and **screenshot the output**:

```
echo ""
echo "╔═══════════════════════════════════════════════╗"
echo "║   BOOTCAMP READINESS CHECK                    ║"
echo "╠═══════════════════════════════════════════════╣"
echo "║                                               ║"
printf "║  OS:        %-28s ║\n" "$(uname -s -r | cut -c1-28)"
printf "║  Docker:    %-28s ║\n" "$(docker --version 2>/dev/null | cut -d'
' -f3 | tr -d ',' || echo 'NOT INSTALLED ❌')"
printf "║  Compose:   %-28s ║\n" "$(docker compose version 2>/dev/null |
cut -d' ' -f4 || echo 'NOT INSTALLED ❌')"
printf "║  Git:       %-28s ║\n" "$(git --version 2>/dev/null | cut -d' ' -
f3 || echo 'NOT INSTALLED ❌')"
printf "║  Node.js:   %-28s ║\n" "$(node --version 2>/dev/null || echo 'NOT
INSTALLED ❌')"
printf "║  pnpm:      %-28s ║\n" "$(pnpm --version 2>/dev/null || echo 'NOT
INSTALLED ❌')"
printf "║  SSH Key:   %-28s ║\n" "$(test -f ~/.ssh/id_ed25519.pub && echo
'EXISTS ✅' || echo 'MISSING ❌')"
printf "║  GitHub:    %-28s ║\n" "$(ssh -T git@github.com 2>&1 | grep -q
'successfully' && echo 'CONNECTED ✅' || echo 'NOT CONNECTED ❌')"
echo "║                                               ║"
echo "╚═══════════════════════════════════════════════╝"

# Docker daemon check
if docker info >/dev/null 2>&1; then
    echo "║  Docker Daemon: RUNNING ✅                    ║"
else
    echo "║  Docker Daemon: NOT RUNNING ❌                ║"
fi

echo "╚═══════════════════════════════════════════════╝"
echo ""
```

Expected Output (All Green):

```
╔═══════════════════════════════════════════════╗
║   BOOTCAMP READINESS CHECK                    ║
╠═══════════════════════════════════════════════╣
║                                               ║
║  OS:        Linux 5.15.x                      ║
║  Docker:    27.x.x                            ║
║  Compose:   v2.x.x                            ║
║  Git:       2.x.x                             ║
║  Node.js:   v20.x.x                           ║
║  pnpm:      9.x.x                             ║
║  SSH Key:   EXISTS ✅                         ║
║  GitHub:    CONNECTED ✅                      ║
║                                               ║
╚═══════════════════════════════════════════════╝
```

```
╔═══════════════════════════════════════╗
║  Docker Daemon: RUNNING ✅            ║
╚═══════════════════════════════════════╝
```

✅ Send the screenshot to the group chat to confirm you're ready!

---

9. Troubleshooting

9.1 SSH: "Permission denied (publickey)"

Problem: `ssh -T git@github.com` returns permission denied.

Fix:

```
# Check if SSH agent has your key loaded
ssh-add -l

# If empty, add your key
ssh-add ~/.ssh/id_ed25519

# Verify the public key is the same one on GitHub
cat ~/.ssh/id_ed25519.pub
```

Then compare the output with what's on https://github.com/settings/keys.

9.2 Docker: "Cannot connect to the Docker daemon"

Windows/WSL:

```
# Start Docker service
sudo service docker start

# Check status
sudo service docker status
```

macOS:

- Make sure Docker Desktop is running (whale icon in menu bar)
- If just installed, restart your Mac

9.3 Docker: "Got permission denied while trying to connect to the Docker daemon socket"

```
# Add yourself to docker group
sudo usermod -aG docker $USER
```

```
# IMPORTANT: Close terminal completely and reopen
# On WSL, run: exit, then open Ubuntu again
```

9.4 WSL: "WslRegisterDistribution failed with error: 0x80370102"

Cause: Virtualization is not enabled in BIOS.

Fix:

1. Restart your computer
2. Enter BIOS/UEFI (usually `F2`, `F12`, `Del`, or `Esc` during boot)
3. Find "Virtualization Technology" or "Intel VT-x" or "AMD-V"
4. Enable it
5. Save and restart

9.5 WSL: "The Windows Subsystem for Linux has not been enabled"

```
# Run in PowerShell as Administrator
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-
Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all
/norestart
```

Restart your computer, then run:

```
wsl --install
```

9.6 Node.js: "command not found" after installation (macOS)

If you installed via Homebrew:

```
echo 'export PATH="/opt/homebrew/opt/node@20/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

9.7 pnpm: "command not found"

```
# Try installing with corepack instead
corepack enable
```

```
corepack prepare pnpm@latest --activate
```

9.8 General: "I can't fix it!"

Fallback plan: If you absolutely cannot get local tools working:

1. Make sure your **GitHub account** is created and SSH key is added
2. Make sure your **DigitalOcean account** is created with payment method
3. On bootcamp day, we'll create your VPS first and you can work directly from there via SSH

The VPS comes with Docker pre-installed, so you'll still be able to follow along.

---

10. Pre-Pull Docker Images (Optional but Recommended)

To save time during the bootcamp (and reduce WiFi load), pre-pull these base images:

```
docker pull node:20-slim
docker pull postgres:17
docker pull caddy:2-alpine
docker pull mcr.microsoft.com/dotnet/aspnet:10.0
docker pull mcr.microsoft.com/dotnet/sdk:10.0
```

This downloads ~2GB total. Better to do this on a good connection before the bootcamp.

Summary — What You Should Have Ready

Item	Status
GitHub account created	☐
GitHub org invitation accepted	☐
DigitalOcean account with payment method	☐
WSL2 + Docker (Windows) OR Docker Desktop (macOS)	☐
Git installed and configured	☐
SSH key generated	☐
SSH key added to GitHub	☐
`ssh -T git@github.com` works	☐
Node.js 20 installed	☐
pnpm installed	☐
Verification script all green	☐

Item	Status
Screenshot sent to group chat	☐

See you Saturday! 🚀

If you have any issues, ask in the group chat immediately — don't wait until bootcamp day.